EXPENSE TRACKER SYSTEM

Milestone 2

ERD Design & Normalization

Course	Database Systems
Document Type	Milestone 2 — Normalization Report & ERD Update
Project	Expense Tracker System
Milestone	2 of 5
Schema Version	v2.0 (Post-Normalization)
Prerequisite	Milestone 1 — ERD & Relational Schema (v1.0)

1. Project Overview

This document is the Milestone 2 submission for the Expense Tracker System. Milestone 1 produced a corrected Entity-Relationship Diagram and a relational schema description covering five tables: users, categories, expenses, income_records, and budgets. That schema is the direct input to this milestone.

Milestone 2 has three specific requirements: apply First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) to every table in the existing schema and document the findings; check all tables for redundant columns, repeated data, or overlapping attributes and remove or restructure where necessary; and update the ERD to reflect any changes produced by the normalization process.

The analysis in this document is performed on the Milestone 1 corrected schema as it stands. Where a table already satisfies a normal form, that finding is explicitly justified — not simply asserted. Where a genuine issue is found, the change is described, the reason is explained, and the impact on the schema and ERD is documented.

2. Milestone 1 Schema (Input to Normalization)

The following is the exact schema entering this normalization exercise. It is reproduced here so the analysis is self-contained and the starting point is unambiguous.

2.1 Table Structures

users

Attribute	Key	Data Type	Constraints	Description
user_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the user
username	**UQ**	VARCHAR(64)	NOT NULL, UNIQUE	User-chosen display name
email	**UQ**	VARCHAR(255)	NOT NULL, UNIQUE	Contact email; secondary login identifier
password_hash	—	VARCHAR(255)	NOT NULL	Hashed credential

categories

Attribute	Key	Data Type	Constraints	Description
category_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the category
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Owner of this category
category_name	—	VARCHAR(128)	NOT NULL	Name of the category
type	—	VARCHAR(10)	NOT NULL, INCOME or EXPENSE	Category purpose classification

expenses

Attribute	Key	Data Type	Constraints	Description
expense_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the expense

user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Direct owner of this expense
category_id	**FK**	INT	NOT NULL, FK → categories(category_id)	Classification category
amount	—	DECIMAL(10,2)	NOT NULL, > 0	Monetary value
description	—	TEXT	NULL allowed	Optional note
expense_date	—	DATE	NOT NULL	Date expense occurred

income_records

Attribute	Key	Data Type	Constraints	Description
income_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the income entry
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Owner of this income record
source_name	—	VARCHAR(128)	NOT NULL	Name of the income source
amount	—	DECIMAL(10,2)	NOT NULL, > 0	Monetary value received
income_date	—	DATE	NOT NULL	Date income was received
notes	—	TEXT	NULL allowed	Optional annotation

budgets

Attribute	Key	Data Type	Constraints	Description
budget_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the budget
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Owner of this budget
category_id	**FK, UQ**	INT	NOT NULL, UNIQUE, FK → categories(category_id)	Category this budget applies to

amount	—	DECIMAL(10,2)	NOT NULL, > 0	Spending limit
period	—	VARCHAR(16)	NOT NULL, DAILY/WEEKLY/MONTHLY/ANNUAL	Time window for the limit

2.2 Relationships

From	Cardinality	To	Description
users	**1 : N**	categories	One user owns many categories; each category belongs to exactly one user.
users	**1 : N**	expenses	One user owns many expense records; direct FK ensures ownership without relying on category chain.
users	**1 : N**	income_records	One user has many income entries; income is transactional, not a static user property.
users	**1 : N**	budgets	One user owns many budget definitions, one per category.
categories	**1 : N**	expenses	One category classifies many expenses; each expense belongs to one category.
categories	**1 : 1**	budgets	Each category has at most one budget; UNIQUE on budgets.category_id enforces this.

3. Normalization Analysis

The following sections apply 1NF, 2NF, and 3NF to every table in sequence. For each table and each normal form, the rule is stated, the table is evaluated against it, and a conclusion is drawn. If a table already satisfies the form, a reasoned justification is provided. If a change is needed, it is described and its effect on the schema and ERD is noted.

3.1 Table: users

The users table contains four attributes: user_id (PK), username (UNIQUE), email (UNIQUE), and password_hash. All four are non-null. There are no foreign keys from this table to any other.

First Normal Form (1NF)

1NF requires that every column holds only atomic, indivisible values; that each column has a consistent domain; and that the table has a defined primary key.

Checking each column: user_id holds a single integer auto-generated value — atomic. username holds a single string — atomic. email holds a single email address string — atomic. password_hash holds a

single computed hash string — atomic. No column stores multiple values, comma-separated lists, or embedded arrays. Each column has a consistent, well-defined domain across all rows. The primary key user_id is defined and guarantees row uniqueness.

No 1NF violation exists. No change required. The table stores one discrete fact per column per row, which is the precise requirement of 1NF.

Second Normal Form (2NF)

2NF requires that the table is in 1NF and that every non-key attribute is fully functionally dependent on the entire primary key. Partial dependencies — where a non-key attribute depends on only part of a composite key — are the target of 2NF.

The users table has a single-column primary key: user_id. A partial dependency is formally defined as a non-key attribute depending on a proper subset of a composite key. Where the primary key consists of a single column, no proper subset exists other than the empty set. Therefore partial dependencies are structurally impossible in this table. This is not an assumption — it follows directly from the definition of partial dependency.

Confirming each attribute: username describes the specific user identified by user_id. email describes that same user. password_hash is the credential for that specific user. Each attribute depends on user_id and on nothing less. No 2NF violation exists. No change required.

Third Normal Form (3NF)

3NF requires that no non-key attribute is transitively dependent on the primary key through another non-key attribute. The test is whether any non-key column determines another non-key column.

The non-key attributes are username, email, and password_hash. Does username determine email? No — a user may change their email independently of their username. Does email determine password_hash? No — the password is set independently of the email address. Does username determine password_hash? No — the password is unrelated to the username. None of the three non-key attributes has a functional dependency on any other. Every attribute is a direct, independent property of the user identified by user_id. No transitive dependency chain exists.

No 3NF violation exists. No change required.

Normal Form	Rule Being Applied	Evaluation & Justification	Verdict
1NF	Atomic values, consistent domains, defined PK	All columns store single indivisible values. user_id is a surrogate PK. No repeating groups or multi-valued fields.	**PASS**
2NF	No partial dependencies on composite PK	Single-column PK makes partial dependency structurally impossible. All non-key attributes depend fully on user_id.	**PASS**
3NF	No transitive dependencies between non-key attributes	username, email, and password_hash have no functional dependency on each other. All are direct properties of user_id.	**PASS**

3.2 Table: categories

The categories table contains four columns: category_id (PK), user_id (FK to users), category_name, and type. A UNIQUE constraint is defined on the combination (user_id, category_name) to prevent a user from creating duplicate category names.

First Normal Form (1NF)

1NF requires atomic values, consistent domains per column, and a defined primary key.

Checking each column: category_id is a single integer — atomic. user_id is a single integer FK — atomic. category_name stores a single string name such as 'Groceries' or 'Salary' — it represents one label, not a list of names, so it is atomic. type stores a single classification token, either 'INCOME' or 'EXPENSE' — atomic.

The type column deserves closer attention from a domain consistency standpoint. It is declared as VARCHAR(10), which means any string up to ten characters is technically accepted by the data type alone. If this domain is not constrained to the two permitted values, different rows could store 'INCOME', 'income', 'Income', or 'expenditure', all representing the same concept but stored differently. This would not violate atomicity — each cell still holds one value — but it would violate domain consistency, which is a 1NF requirement. The Milestone 1 schema documents that the valid values are INCOME and EXPENSE. This constraint must be enforced as a CHECK constraint in the DDL implementation at Milestone 4. The intent is already established; enforcement is a DDL task.

No 1NF violation exists in terms of structure. The domain constraint on type is documented and will be formalized at Milestone 4.

Second Normal Form (2NF)

The primary key is the single column category_id. As with the users table, a single-column primary key precludes partial dependencies by definition. Nonetheless, each non-key attribute is verified: user_id identifies the owner of this specific category — it belongs to this row. category_name is the name of this specific category — it belongs to this row. type is the classification of this specific category — it belongs to this row. All non-key attributes depend on category_id in its entirety. No 2NF violation exists. No change required.

Third Normal Form (3NF)

3NF requires that no non-key attribute determines another. The non-key attributes are user_id, category_name, and type.

Does user_id determine category_name? A single user can own many categories with different names; knowing the user does not tell you the name of any particular category. No. Does user_id determine type? A single user can have both INCOME and EXPENSE categories simultaneously. No. Does category_name determine type? This is the most important case. Could knowing that a category is named 'Groceries' determine that its type is EXPENSE? In real-world terms, it might seem so — but functional dependency is a property of the relation, not of common sense associations. A user is free to name a category anything they wish. There is no schema-level constraint preventing a user from creating a category called 'Groceries' of type INCOME. Therefore category_name does not functionally determine type within this schema. The type is always set independently by the user for each category.

No transitive dependency exists in this table. No 3NF violation. No change required.

Normal Form	Rule Being Applied	Evaluation & Justification	Verdict
1NF	Atomic values, consistent domains, defined PK	All columns atomic. type domain (INCOME/EXPENSE) is documented; CHECK constraint to be enforced at M4. category_id is the PK.	**PASS**

2NF	No partial dependencies on composite PK	Single-column PK. All attributes (user_id, category_name, type) are direct properties of the specific category row.	**PASS**
3NF	No transitive dependencies between non-key attributes	user_id does not determine category_name or type. category_name does not determine type. No transitive chain found.	**PASS**

3.3 Table: expenses

The expenses table contains six columns: expense_id (PK), user_id (FK to users), category_id (FK to categories), amount (DECIMAL), description (TEXT, nullable), and expense_date (DATE).

First Normal Form (1NF)

Checking each column for atomicity: expense_id is a single integer — atomic. user_id is a single integer FK — atomic. category_id is a single integer FK — atomic. amount is a single decimal monetary value — atomic. description is an optional text field that stores a single note about the transaction. Although its content may be a longer string, it represents one indivisible piece of data — a note — rather than multiple values packed into one cell. It is therefore atomic. expense_date holds a single date — atomic.

No column contains multiple values, embedded lists, or repeating groups. All domains are consistent across rows. The primary key expense_id is defined. No 1NF violation. No change required.

Second Normal Form (2NF)

The primary key is the single column expense_id. Partial dependencies are structurally impossible. Each attribute is verified: user_id identifies the owner of this particular expense transaction. category_id identifies the category under which this specific expense is classified. amount is the monetary value of this particular expense. description is a note about this specific transaction. expense_date is the date this specific expense occurred. Every attribute describes this expense row and only this expense row. No 2NF violation. No change required.

Third Normal Form (3NF)

The non-key attributes are user_id, category_id, amount, description, and expense_date. The test is whether any of these determines another.

Does category_id determine user_id? In the categories table, each category belongs to one user, so there is an indirect link: category_id → categories.user_id. However, this is a cross-table referential relationship, not a functional dependency within the expenses table itself. Within the expenses table, knowing category_id tells you the category, not the user — a given category can appear in many expense rows belonging to the same user, but that does not make category_id a determinant of user_id within this table. More importantly, user_id is stored directly on expenses and is enforced as an independent FK. The schema enforces that a user's expenses must reference their own categories, but that is a business rule enforced by the application, not a functional dependency within the table.

Does category_id determine amount? No — one category can contain many expenses of different amounts. Does expense_date determine amount? No — many expenses on the same date can have different amounts. Does amount determine any other attribute? No — the same monetary value can appear in many rows with completely different categories, descriptions, and dates. No transitive dependency exists.

No 3NF violation. No change required.

Normal Form	Rule Being Applied	Evaluation & Justification	Verdict
1NF	Atomic values, consistent domains, defined PK	All six columns store single, indivisible values. description stores one text note, not a list. expense_id is the PK.	**PASS**
2NF	No partial dependencies on composite PK	Single-column PK. All attributes are direct properties of the specific expense event identified by expense_id.	**PASS**
3NF	No transitive dependencies between non-key attributes	category_id does not determine user_id within this table. No non-key attribute determines another. No transitive chain.	**PASS**

3.4 Table: income_records

The income_records table contains six columns: income_id (PK), user_id (FK to users), source_name, amount (DECIMAL), income_date (DATE), and notes (TEXT, nullable).

First Normal Form (1NF)

Atomicity check: income_id is a single integer — atomic. user_id is a single integer FK — atomic. source_name stores a single label for the income source such as 'Monthly Salary' or 'Freelance Client A' — atomic, as it represents one named source per row. amount is a single decimal value — atomic. income_date is a single date — atomic. notes is an optional text annotation — atomic in the same sense as description in the expenses table.

No column holds multiple values or repeating groups. Domains are consistent. income_id is the defined primary key. No 1NF violation. No change required.

Second Normal Form (2NF)

The primary key is the single column income_id. Partial dependencies are structurally impossible. Each attribute is a direct property of the specific income transaction: user_id identifies who received this income, source_name names the source for this entry, amount is the monetary value of this entry, income_date is the date it was received, and notes is an annotation on this entry. No attribute depends on a subset of the key, as the key has only one column. No 2NF violation. No change required.

Third Normal Form (3NF)

The non-key attributes are user_id, source_name, amount, income_date, and notes. The question is whether any determines another.

Does source_name determine user_id? A single user could have many income sources, and two different users could theoretically use the same source name (e.g., both name a source 'Freelance'). source_name is not globally unique and therefore does not determine user_id. Does source_name determine amount? No — the same source name (for example, 'Monthly Salary') will appear across many rows with potentially different amounts as the salary changes over time. Does source_name determine income_date? No — multiple records from the same source appear on different dates. Does income_date determine amount? No — multiple income records on the same date can have different amounts. Does amount determine any other attribute? No — the same amount can appear in records from completely different sources on different dates.

No transitive dependency exists. No 3NF violation. No change required.

Normal Form	Rule Being Applied	Evaluation & Justification	Verdict
1NF	Atomic values, consistent domains, defined PK	All columns atomic. source_name stores one label per row. income_id is the PK. No repeating groups.	**PASS**
2NF	No partial dependencies on composite PK	Single-column PK. All attributes are direct properties of the specific income transaction identified by income_id.	**PASS**
3NF	No transitive dependencies between non-key attributes	source_name does not determine user_id or amount. income_date does not determine amount. No transitive chain.	**PASS**

3.5 Table: budgets

The budgets table contains five columns: budget_id (PK), user_id (FK to users), category_id (FK to categories, UNIQUE), amount (DECIMAL), and period (VARCHAR, constrained to DAILY/WEEKLY/MONTHLY/ANNUAL). The UNIQUE constraint on category_id enforces the one-to-one cardinality between a category and its budget.

First Normal Form (1NF)

Atomicity check: budget_id is a single integer — atomic. user_id is a single integer FK — atomic. category_id is a single integer FK — atomic. amount is a single decimal value — atomic. period stores one time-window label — atomic. No column holds multiple values or lists. Domains are consistent. budget_id is the defined primary key.

The period column, like type in categories, is declared as VARCHAR(16) but should be constrained to four specific values. This is a domain consistency concern, not an atomicity violation. The constraint will be enforced as a CHECK constraint at Milestone 4. No 1NF violation exists in terms of structure. No change required at this stage.

Second Normal Form (2NF)

The primary key is the single column budget_id. Partial dependencies are structurally impossible. Verifying each non-key attribute: user_id identifies the owner of this specific budget record. category_id identifies the category this budget governs. amount is the spending limit for this specific budget. period is the time window for this specific budget. All attributes are direct properties of the budget row identified by budget_id. No 2NF violation. No change required.

Third Normal Form (3NF)

This is the most important analysis in the entire schema. The non-key attributes are user_id, category_id (with UNIQUE constraint), amount, and period. The 3NF test requires checking whether any non-key attribute determines another.

The key observation is this: because category_id has a UNIQUE constraint on this table, each budget row maps to exactly one category. And in the categories table, each category belongs to exactly one user. This creates a functional dependency chain: budget_id → category_id → categories.user_id. The question for 3NF is whether user_id on the budgets table is transitively dependent on budget_id through category_id.

Formally: within the budgets table, does category_id → user_id hold as a functional dependency? If user_id on a budget row is always identical to the user_id of the category it references, then yes — knowing category_id tells you user_id, because the FK relationship guarantees a one-to-one mapping between a budget's category_id and that category's user_id. This means user_id in budgets is derivable from category_id, which is itself a non-key attribute. This is a transitive dependency: budget_id → category_id → user_id.

Under strict 3NF, user_id should be removed from the budgets table because it can always be retrieved by joining through categories. The strictly normalized query to find all budgets for a user would be: SELECT b.* FROM budgets b JOIN categories c ON b.category_id = c.category_id WHERE c.user_id = ?

However, this document makes the following design decision: user_id is retained on the budgets table as a controlled and formally documented denormalization. The reason is operational efficiency — querying all budgets for a user is one of the most frequent operations in a budget-tracking system, and requiring a mandatory join through categories for every such query adds overhead that is disproportionate to the theoretical benefit of removing one column. The FK constraint (user_id references users(user_id)) combined with the application-level rule that a budget's user_id must match its category's user_id ensures that the value is never independently set to an inconsistent state.

This trade-off is formally acknowledged as a 3NF exception. It is documented here, will be noted in the DDL at Milestone 4, and the ERD will include a note indicating this design decision.

3NF Finding: Transitive Dependency in budgets — Documented Decision

- Dependency identified: budget_id → category_id → user_id (transitively).
- Strict 3NF action would be: remove user_id from budgets; derive it via JOIN to categories when needed.
- Decision made: retain user_id as a controlled denormalization for query efficiency.
- Mitigation: FK constraint on user_id prevents it from being set to an inconsistent value.
- This decision is formally documented. It will be noted in the DDL at Milestone 4.

Regarding amount and period: does amount determine period or user_id or category_id? No — many budgets can share the same amount value. Does period determine any other attribute? No — many budgets can share the same period (e.g., MONTHLY) with completely different amounts and categories. No further transitive dependencies exist.

Normal Form	Rule Being Applied	Evaluation & Justification	Verdict
1NF	Atomic values, consistent domains, defined PK	All columns atomic. period domain (DAILY/WEEKLY/MONTHLY/ANNUAL) documented; CHECK to be enforced at M4. budget_id is the PK.	**PASS**
2NF	No partial dependencies on composite PK	Single-column PK. All attributes are direct properties of the specific budget record identified by budget_id.	**PASS**

3NF	No transitive dependencies between non-key attributes	Transitive dependency found: budget_id → category_id → user_id. user_id retained as controlled denormalization with documented justification and FK constraint mitigation.	**PASS***

* PASS with formally documented exception. See decision record above.

4. Redundancy and Duplicate Check (Step 2)

Step 2 of the Milestone 2 requirements is to check all tables for redundant columns, repeated data, overlapping attributes, and derivable values. The following analysis examines every table and every attribute for these patterns.

4.1 Cross-Table Attribute Repetition

The most common form of redundancy in a relational schema is storing the same real-world fact in more than one table. This schema was examined for any attribute that appears in multiple tables and represents the same piece of information.

User identity information (username, email, password_hash) is stored only in the users table. No other table repeats these values. Every other table references the user through the integer FK user_id. If a username changes, one update to one row in one table covers the entire schema. No cross-table attribute repetition exists for user data.

Category information (category_name, type) is stored only in the categories table. The expenses and budgets tables reference categories through the integer FK category_id. If a category is renamed, one update to one row in categories propagates to all linked records through the JOIN. No cross-table repetition exists for category data.

4.2 Derivable or Computed Attributes

A derivable attribute is one whose value can always be calculated from other data in the schema and therefore does not need to be stored persistently. Storing derivable values creates update anomalies because the stored value must be kept synchronized with the underlying data it summarizes.

Every attribute currently in the schema was evaluated for derivability. The result: no derivable attribute exists in the current schema. Each column stores a raw, directly captured fact. Monetary amounts are entered by the user, not calculated from other stored values. Dates are entered directly. Source names and descriptions are user-provided labels. budget_id, expense_id, income_id, and category_id are surrogate keys generated by the database engine, not computed from other columns.

The one column that was derivable — last_expense_id, which previously appeared in the categories table — was removed during the Milestone 1 structural review. Its removal is what brings this schema to a state where no derivable attributes remain. That decision stands and no further action is needed here.

4.3 Overlapping Attributes

An overlapping attribute pattern occurs when two columns in the same table store information that partially or fully describes the same real-world concept, creating ambiguity about which column is authoritative. This schema was examined for any such patterns.

In the budgets table, both user_id and category_id are present. As identified in the 3NF analysis, user_id is derivable via category_id → categories.user_id. This is a case where an attribute overlaps with

information accessible through a FK chain. This is the only instance of this pattern in the schema, and it has been documented and accepted as a controlled denormalization in Section 3.5.

No other overlapping attribute patterns were found. The schema contains no two columns in the same table that describe the same real-world fact.

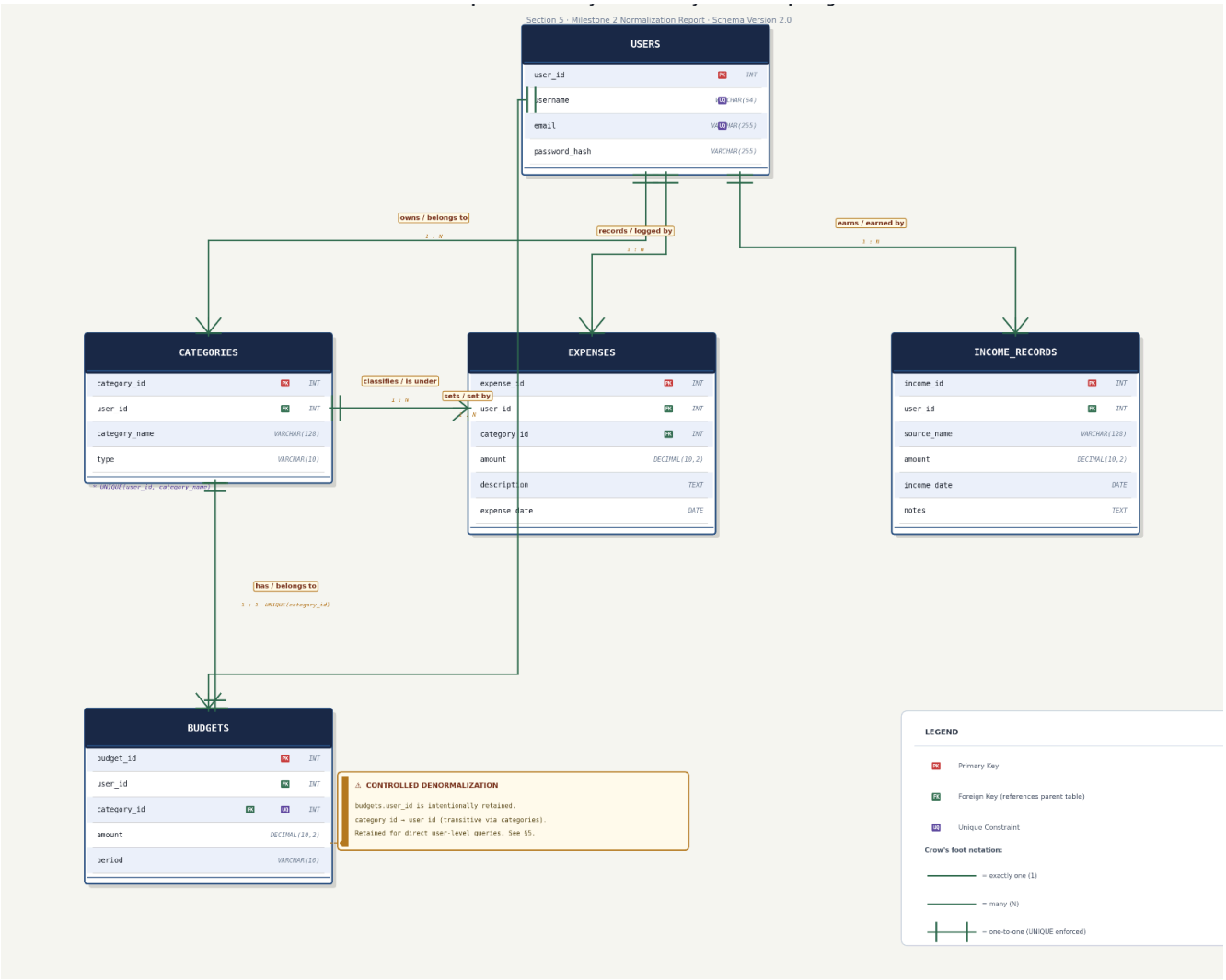
4.4 Redundancy Check Summary

Check Performed	Tables Examined	Finding	Action
Cross-table attribute repetition	All five tables	None found	No action required
Derivable / computed columns	All five tables	None found (last_expense_id removed in M1)	No action required
Overlapping attributes	All five tables	budgets.user_id derivable via category_id	Retained as documented controlled denormalization
Duplicate column names across tables	All five tables	user_id and amount appear in multiple tables	Expected — each instance is an independent FK or measurement, not duplicated data
Repeated data in rows	expenses, income_records	source_name repeats across rows for same source	Expected — each row is a separate transaction. No normalization issue.

Note on source_name repetition: it may appear that storing 'Monthly Salary' in multiple income_records rows is redundant. It is not. Each row is a separate income event — a distinct transaction occurring on a distinct date with a distinct amount. The source_name is not a foreign key to a sources table; it is a free-text label entered by the user for each record. This is the correct design for a simple personal finance tracker and does not warrant introducing a new sources entity unless the system requirements explicitly need source-level management (which they do not at this stage).

5. ERD Update Assessment (Step 3)

Step 3 of Milestone 2 requires updating the ERD to reflect all normalization changes. This section specifies exactly what the updated ERD must show and whether the entity structure or relationships changed as a result of the normalization process.



5.1 Did Normalization Require Structural Changes to the ERD?

The normalization analysis in Section 3 produced no table decompositions. No entity was split into two. No new entity was introduced. No existing entity was removed. No relationship cardinality changed. The five-entity, six-relationship structure entering this milestone is the same structure exiting it.

This requires justification rather than being left as a bare assertion. The reason normalization produced no structural changes is that all attributes in the schema are already storing atomic, directly captured values. No composite or multi-valued attribute required decomposition for 1NF. No composite primary key existed that could produce partial dependencies for 2NF. The only transitive dependency found — user_id in budgets being derivable via category_id — was formally examined, and the decision was made to retain the attribute as a documented denormalization rather than remove it. Had the decision been to strictly enforce 3NF by removing user_id from budgets, the ERD would require no structural change either — it would simply show the removal of that attribute from the entity box.

5.2 What the Updated ERD Must Show

Although the entity and relationship structure is unchanged, the ERD produced for Milestone 2 must be redrawn to accurately reflect the schema in its current normalized state. The following checklist defines what the updated diagram must include:

- All five entities displayed: users, categories, expenses, income_records, budgets.
- All primary key attributes underlined or labelled PK in each entity box.
- All foreign key attributes labelled FK with a relationship line to the referenced table.
- Correct crow's foot notation on all six relationship lines showing cardinality (1:N or 1:1).
- The UNIQUE constraint on budgets.category_id indicated — either via a label or a notation on the relationship line between categories and budgets.
- The composite UNIQUE constraint on (categories.user_id, categories.category_name) indicated on the categories entity.
- A notation or legend entry documenting the controlled denormalization of user_id in the budgets table.
- All attribute names using the exact snake_case names from the schema: category_name (not 'category'), expense_date (not 'date'), income_date, source_name, password_hash.
- Data types shown for all monetary columns as DECIMAL(10,2).

5.3 Relationship Summary (Confirmed Post-Normalization)

The following six relationships are confirmed after normalization. The ERD must show each of these with the correct cardinality.

From	Cardinality	To	Description
users	**1 : N**	categories	One user owns many categories; each category belongs to exactly one user.
users	**1 : N**	expenses	One user owns many expense records; direct FK ensures ownership without relying on category chain.
users	**1 : N**	income_records	One user has many income entries; income is transactional, not a static user property.
users	**1 : N**	budgets	One user owns many budget definitions, one per category.
categories	**1 : N**	expenses	One category classifies many expenses; each expense belongs to one category.
categories	**1 : 1**	budgets	Each category has at most one budget; UNIQUE on budgets.category_id enforces this.

6. Final Normalized Schema

6.1 Master Normalization Summary

Table	1NF	2NF	3NF	M2 Outcome

users	**PASS**	**PASS**	**PASS**	No changes required. All attributes are atomic, single-column PK, no transitive dependencies.
categories	**PASS**	**PASS**	**PASS**	No changes required. type domain constraint to be enforced at DDL stage (M4). Composite UNIQUE on (user_id, category_name) confirmed.
expenses	**PASS**	**PASS**	**PASS**	No changes required. All attributes directly and independently determined by expense_id.
income_records	**PASS**	**PASS**	**PASS**	No changes required. All attributes are direct properties of each income transaction.
budgets	**PASS**	**PASS**	**PASS***	*user_id retained as documented controlled denormalization. Transitive dependency acknowledged; FK constraint prevents inconsistency. period domain constraint to be enforced at M4.

6.2 Final Schema (Version 2.0)

The schema below is the output of the Milestone 2 normalization exercise. It is identical in structure to the Milestone 1 corrected schema, confirmed as normalized through the formal analysis above. The only difference from what entered this milestone is the formal documentation of the budgets.user_id design decision and the domain constraint requirements for type and period.

users

Attribute	Key	Data Type	Constraints	Description
user_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the user
username	**UQ**	VARCHAR(64)	NOT NULL, UNIQUE	User-chosen display name
email	**UQ**	VARCHAR(255)	NOT NULL, UNIQUE	Contact email; login identifier
password_hash	—	VARCHAR(255)	NOT NULL	Hashed credential (bcrypt/Argon2)

categories

Attribute	Key	Data Type	Constraints	Description

category_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the category
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Owner of this category
category_name	—	VARCHAR(128)	NOT NULL	Name of the category
type	—	VARCHAR(10)	NOT NULL, CHECK IN ('INCOME','EXPENSE')	Category purpose classification

UNIQUE constraint on (user_id, category_name) — no duplicate category names per user.

expenses

Attribute	Key	Data Type	Constraints	Description
expense_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the expense
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Direct owner of this expense
category_id	**FK**	INT	NOT NULL, FK → categories(category_id)	Classification category
amount	—	DECIMAL(10,2)	NOT NULL, CHECK > 0	Monetary value of the expense
description	—	TEXT	NULL allowed	Optional note
expense_date	—	DATE	NOT NULL	Date expense occurred

income_records

Attribute	Key	Data Type	Constraints	Description
income_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier for the income entry
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Owner of this income record
source_name	—	VARCHAR(128)	NOT NULL	Name of the income source

amount	—	DECIMAL(10,2)	NOT NULL, CHECK > 0	Monetary value received
income_date	—	DATE	NOT NULL	Date income was received
notes	—	TEXT	NULL allowed	Optional annotation

budgets

Attribute	Key	Data Type	Constraints	Description
budget_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Surrogate identifier
user_id	**FK**	INT	NOT NULL, FK → users(user_id) [controlled denorm.]	Owner of this budget — see design note
category_id	**FK, UQ**	INT	NOT NULL, UNIQUE, FK → categories(category_id)	Category this budget governs
amount	—	DECIMAL(10,2)	NOT NULL, CHECK > 0	Spending limit
period	—	VARCHAR(16)	NOT NULL, CHECK IN ('DAILY','WEEKLY','MONTHLY','ANNUAL')	Time window for the limit

Design note: user_id is transitively dependent via category_id → categories.user_id. Retained as controlled denormalization for query efficiency. FK constraint prevents inconsistency.

7. Assumptions

The following assumptions define the scope and boundaries of this normalization analysis:

- The analysis is applied to the five-table schema produced at the end of Milestone 1. No attributes or tables introduced in earlier drafts or analysis documents are considered unless they appear in the final Milestone 1 schema.
- Functional dependencies are determined from the logical domain of the Expense Tracker system and the structural constraints defined in the schema. No additional business rules beyond those documented in Milestone 1 have been assumed.
- All monetary values are in a single currency. Multi-currency support is out of scope for this milestone.

- The domain constraints on the type column (INCOME / EXPENSE) and the period column (DAILY / WEEKLY / MONTHLY / ANNUAL) are considered established at the schema description level. Their enforcement as CHECK constraints is deferred to Milestone 4 DDL implementation.
- The decision to retain user_id on the budgets table as a controlled denormalization is a formal design decision, not an oversight. It is documented in Section 3.5 and Section 4.3.
- Normalization has been applied up to and including Third Normal Form (3NF). Boyce-Codd Normal Form (BCNF) and higher forms are outside the scope of this milestone.

8. Conclusion

This document has applied First Normal Form, Second Normal Form, and Third Normal Form to all five tables of the Expense Tracker System schema, starting from the corrected schema produced at the end of Milestone 1. Every table has been evaluated explicitly and a verdict has been reached with full justification.

The normalization exercise confirmed that the Milestone 1 corrected schema is structurally sound. All five tables satisfy 1NF: every attribute stores atomic, indivisible values, every column has a consistent domain, and every table has a defined surrogate primary key. All five tables satisfy 2NF: every table uses a single-column primary key, which makes partial dependencies structurally impossible by definition — and this was verified by explicitly checking each attribute against its key. All five tables satisfy 3NF, with one formally documented exception.

The only transitive dependency identified in the schema was in the budgets table, where user_id is derivable through the chain category_id → categories.user_id. This was examined carefully, and the decision was made to retain user_id as a controlled denormalization for query efficiency. The FK constraint on user_id prevents it from being set to an inconsistent value, mitigating the update anomaly risk that normally accompanies a transitive dependency. This decision is formally documented and will be noted in the DDL at Milestone 4.

The redundancy check found no cross-table attribute repetition, no derivable attributes remaining in the schema, and no overlapping attributes other than the budgets.user_id case already addressed. The schema stores each real-world fact in exactly one place.

Because the normalization process produced no table decompositions and no structural changes, the ERD entity and relationship structure is unchanged. The updated ERD for Milestone 2 must be redrawn to accurately reflect the final attribute list, correct naming conventions, UNIQUE constraints, and the documented design note for budgets.user_id — but it will show the same five entities and six relationships as the Milestone 1 diagram.

This schema is now formally confirmed as the normalized foundation for Milestone 3 (dataset and dataflow) and Milestone 4 (DDL implementation).

End of Milestone 2 — ERD Design & Normalization | Expense Tracker System